Trabajo Práctico · Unidad 7: NoSQL y Nuevas Formas de Almacenamiento

Food Store — Modelado Documental en MongoDB, Validación Cruzada y Evaluación Tecnológica

Cátedra: Bases de Datos II — Tecnicatura Universitaria en Programación (UTN)
Integrantes: Tiziano Valentini y Juan Martin Garcia
Repositorio Institucional: `Trabajos-Practicos-BD-Valentini-Garcia`
Base Relacional de Origen: `food_store_dev` (PostgreSQL 18)
Base Documental de Destino: `food_store_nosql` (MongoDB 9.0)

Parte 1 · Modelo Documental y Migración

1. Tablas y Claves de PostgreSQL Intervinientes (Consigna 1)

El subconjunto seleccionado para el modelado documental corresponde al núcleo transaccional de ventas de Food Store: **Pedidos y Detalles de Pedido**, interactuando directamente con Clientes, Productos y Categorías. En el esquema relacional (`TP1_FoodStore/schema.sql` y `TP5_Indices_Vistas/soft_delete.sql`), las tablas y restricciones que intervienen son:

1. **`Pedido`**:
   - **Clave Primaria (PK):** `id_pedido` (`BIGINT GENERATED ALWAYS AS IDENTITY`).
   - **Clave Foránea (FK):** `id_cliente` $\rightarrow$ `Cliente(id_cliente)` con regla `ON DELETE RESTRICT` (impide eliminar clientes con historial de comprobantes asociados).
   - **Atributos:** `fecha_hora` (`TIMESTAMPTZ`), `forma_pago` (`forma_pago_enum`: `'EFECTIVO'`, `'TARJETA'`, `'TRANSFERENCIA'`, `'BILLETERA_DIGITAL'`), `estado` (`VARCHAR(20)`: `'ENTREGADO'`, `'EN_PREPARACION'`, `'PENDIENTE'`), `eliminado` (`BOOLEAN NOT NULL DEFAULT FALSE` para soft delete).
2. **`Detalle_Pedido`** (Entidad asociativa $N : M$):
   - **Clave Primaria Compuesta (PK):** `(id_pedido, id_producto)`.
   - **Claves Foráneas (FK):** `id_pedido` $\rightarrow$ `Pedido(id_pedido)` con `ON DELETE CASCADE` (la anulación física del comprobante arrastra sus líneas), e `id_producto` $\rightarrow$ `Producto(id_producto)` con `ON DELETE RESTRICT`.
   - **Restricciones de Dominio:** `chk_detalle_cantidad_positiva` (`CHECK (cantidad > 0)`) y `chk_detalle_precio_no_negativo` (`CHECK (precio_unitario >= 0)`).
   - **Auditoría y Vigencia:** `eliminado` (`BOOLEAN DEFAULT FALSE`) y `stock_descontado` (`BOOLEAN DEFAULT FALSE`).
3. **`Cliente`**:
   - **Clave Primaria (PK):** `id_cliente` (`BIGINT`).
   - **Clave Candidata:** `correo` (`VARCHAR(150)` con índice único parcial `uq_cliente_correo_vigente WHERE eliminado = FALSE`).
   - **Atributos:** `nombre`, `telefono`, `eliminado`, `created_at`.
4. **`Producto`** y **`Categoria`**:

- **Producto:** PK `id_producto`, FK `id_categoria` → `Categoria(id_categoria) ON DELETE RESTRICT`, `nombre_producto`, `precio_actual`, `stock`, `activo`.
- **Categoria:** PK `id_categoria`, `nombre_categoria`, `descripcion`.

---

2. Esquema Formal de la Colección Documental `pedidos`

En MongoDB, la colección adopta el patrón de diseño basado en la **Raíz del Agregado (Aggregate Root)** de Domain-Driven Design (DDD). Cada documento almacena de forma autosuficiente la cabecera, la instantánea del cliente y el arreglo de líneas:

```
{
  "_id": "ObjectId",
  "id_pedido": "Long / Int (Identificador conservado de PostgreSQL, índice
UNIQUE)",
  "fecha_hora": "ISODate (Timestamp de la transacción)",
  "forma_pago": "String ('EFECTIVO' | 'TARJETA' | 'TRANSFERENCIA' |
'BILLETERA_DIGITAL')",
  "estado": "String ('ENTREGADO' | 'EN_PREPARACION' | 'PENDIENTE' | 'CANCELADO')",
  "eliminado": "Boolean (Bandera de soft delete)",
  "cliente": {
    "id_cliente": "Long / Int (Referencia relacional de origen)",
    "nombre": "String",
    "email": "String"
  },
  "lineas": [
    {
      "id_producto": "Long / Int",
      "nombre_producto": "String",
      "categoria": "String (Desnormalizada desde Categoria)",
      "cantidad": "Int (Entero positivo > 0)",
      "precio_unitario": "Double / Decimal128 (Precio histórico pactado)",
      "subtotal": "Double / Decimal128 (cantidad * precio_unitario)"
    }
  ],
  "total_pedido": "Double / Decimal128 (Consistente con la suma de subtotales)"
}
```

---

3. Evidencia Real de Inserción `insertMany()` (Consigna 2)

Se migraron 6 pedidos reales desde la base relacional `food_store_dev`, ejecutando el script `consultas_mongodb.js` en `mongosh`. La salida obtenida en terminal certifica la persistencia atómica:

```
================================================================================
 >>> SELECCIONANDO BASE DE DATOS: food_store_nosql
================================================================================

 >>> EVIDENCIA DE INSERCIÓN (insertMany):
{
```

```
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6ae5114bf358e80f913406d8'),
      '1': ObjectId('6ae5114bf358e80f913406d9'),
      '2': ObjectId('6ae5114bf358e80f913406da'),
      '3': ObjectId('6ae5114bf358e80f913406db'),
      '4': ObjectId('6ae5114bf358e80f913406dc'),
      '5': ObjectId('6ae5114bf358e80f913406dd')
    }
  }
   >>> Total documentos insertados: 6
```

---

4. Reglas de Integridad Conservadas y Justificación de Embedding (Consigna 3)

A. Dos Reglas de Integridad Conservadas:

1. **Inmutabilidad y congelamiento del precio histórico de venta:** En el esquema de PostgreSQL, la tabla `Detalle_Pedido` almacena `precio_unitario`, independizándolo de las fluctuaciones en `Producto.precio_actual`. Durante la migración documental se preservó este valor histórico exacto en cada ítem de `lineas`. Esto garantiza que eventuales aumentos o bajas en el catálogo de productos no alteren retroactivamente el monto facturado ni las estadísticas contables pasadas.
2. **Consistencia matemática y cardinalidad estricta del agregado:** Se conservó la regla de negocio donde `total_pedido = SUM(cantidad * precio_unitario)`. Además, se validó que cada línea conserve una cantidad estrictamente positiva (`cantidad > 0`), respetando la restricción `chk_detalle_cantidad_positiva` de PostgreSQL.

B. Justificación Técnica de la Estrategia de Embebido (Embedding vs Referencing):

- **Líneas del pedido embebidas (`lineas: [...]`):** Una línea de pedido no posee existencia conceptual independiente; su ciclo de vida está 100% subordinado al pedido. En sistemas relacionales, su separación en `Detalle_Pedido` obedece a la Primera Forma Normal ($1FN$), obligando a costosos `JOIN` en cada lectura. En MongoDB, el embebido garantiza **localidad de referencia espacial en disco** (los datos se leen contiguos en una única operación de I/O de WiredTiger) y **atomicidad a nivel de documento único** (*single-document atomicity*), eliminando la necesidad de transacciones distribuidas o bloqueos complejos para crear o consultar un pedido con todos sus ítems.
- **Datos esenciales del cliente embebidos (`cliente: { id_cliente, nombre, email }`):** Se adoptó el patrón de *Desnormalización Deliberada por Snapshot*. Un comprobante de compra debe reflejar los datos del cliente al instante exacto de la operación. Si en el futuro el cliente actualiza su razón social o correo en la colección de usuarios, el pedido histórico debe mantener inalterada la foto fiscal con la que fue emitido y despachado.

Parte 2 · Consultas y Verificación Cruzada

1. Ejecución de Consultas en MongoDB (`mongosh` — Consigna 4)

Consulta (a): Mostrar todos los pedidos con identificador y estado

```
db.pedidos.find(
  {},
  { id_pedido: 1, estado: 1, forma_pago: 1, total_pedido: 1, _id: 0 }
).sort({ id_pedido: 1 });
```

Salida de consola (mongosh):

```
[
  { "id_pedido": 1, "forma_pago": "EFECTIVO", "estado": "ENTREGADO",
"total_pedido": 17804.34 },
  { "id_pedido": 2, "forma_pago": "TARJETA", "estado": "ENTREGADO",
"total_pedido": 17662.15 },
  { "id_pedido": 3, "forma_pago": "TRANSFERENCIA", "estado": "EN_PREPARACION",
"total_pedido": 12539.38 },
  { "id_pedido": 4, "forma_pago": "TARJETA", "estado": "ENTREGADO",
"total_pedido": 3175.7 },
  { "id_pedido": 5, "forma_pago": "BILLETERA_DIGITAL", "estado": "ENTREGADO",
"total_pedido": 25104.04 },
  { "id_pedido": 6, "forma_pago": "TRANSFERENCIA", "estado": "PENDIENTE",
"total_pedido": 24299.84 }
]
// Total recuperados: 6
```

Consulta (b): Filtrar pedidos por estado existente (`estado: "ENTREGADO"`)

```
db.pedidos.find(
  { estado: "ENTREGADO" },
  { id_pedido: 1, fecha_hora: 1, forma_pago: 1, estado: 1, "cliente.nombre": 1,
total_pedido: 1, _id: 0 }
).sort({ id_pedido: 1 });
```

Salida de consola (mongosh):

```
[
  { "id_pedido": 1, "fecha_hora": ISODate("2026-09-21T22:22:22.225Z"),
"forma_pago": "EFECTIVO", "estado": "ENTREGADO", "cliente": { "nombre": "Ana
Gómez" }, "total_pedido": 17804.34 },
  { "id_pedido": 2, "fecha_hora": ISODate("2026-09-26T22:22:22.225Z"),
"forma_pago": "TARJETA", "estado": "ENTREGADO", "cliente": { "nombre": "Luis Paz"
}, "total_pedido": 17662.15 },
  { "id_pedido": 4, "fecha_hora": ISODate("2025-01-05T17:38:06.391Z"),
"forma_pago": "TARJETA", "estado": "ENTREGADO", "cliente": { "nombre": "Cliente
7650" }, "total_pedido": 3175.7 },
  { "id_pedido": 5, "fecha_hora": ISODate("2026-08-29T11:38:48.485Z"),
"forma_pago": "BILLETERA_DIGITAL", "estado": "ENTREGADO", "cliente": { "nombre":
```

```
"Cliente 19190" }, "total_pedido": 25104.04 }
]
// Total recuperados: 4 (IDs: 1, 2, 4, 5)
```

Consulta (c): Buscar pedidos que contengan un producto determinado (`id_producto = 3`, "Coca 1.5L")

```
db.pedidos.find(
  { "lineas.id_producto": 3 },
  {
    id_pedido: 1,
    fecha_hora: 1,
    estado: 1,
    "cliente.nombre": 1,
    lineas: { $elemMatch: { id_producto: 3 } },
    total_pedido: 1,
    _id: 0
  }
).sort({ id_pedido: 1 });
```

Salida de consola (`mongosh`):

```
[
  {
    "id_pedido": 1,
    "fecha_hora": ISODate("2026-09-21T22:22:22.225Z"),
    "estado": "ENTREGADO",
    "cliente": { "nombre": "Ana Gómez" },
    "total_pedido": 17804.34,
    "lineas": [{ "id_producto": 3, "nombre_producto": "Coca 1.5L", "categoria":
"Bebidas", "cantidad": 1, "precio_unitario": 800, "subtotal": 800 }]
  },
  {
    "id_pedido": 3,
    "fecha_hora": ISODate("2026-09-30T22:22:22.225Z"),
    "estado": "EN_PREPARACION",
    "cliente": { "nombre": "Marta Ruiz" },
    "total_pedido": 12539.38,
    "lineas": [{ "id_producto": 3, "nombre_producto": "Coca 1.5L", "categoria":
"Bebidas", "cantidad": 4, "precio_unitario": 800, "subtotal": 3200 }]
  }
]
// Total recuperados: 2 (IDs: 1, 3)
```

Consulta (d): Buscar pedidos con total superior a un importe justificado (`total_pedido > 15000.00`)

```
db.pedidos.find(
  { total_pedido: { $gt: 15000.00 } },
  { id_pedido: 1, estado: 1, "cliente.nombre": 1, total_pedido: 1, _id: 0 }
).sort({ total_pedido: -1 });
```

Salida de consola (mongosh):

```
[
  { "id_pedido": 5, "estado": "ENTREGADO", "cliente": { "nombre": "Cliente 19190"
}, "total_pedido": 25104.04 },
  { "id_pedido": 6, "estado": "PENDIENTE", "cliente": { "nombre": "Cliente 14469"
}, "total_pedido": 24299.84 },
  { "id_pedido": 1, "estado": "ENTREGADO", "cliente": { "nombre": "Ana Gómez" },
"total_pedido": 17804.34 },
  { "id_pedido": 2, "estado": "ENTREGADO", "cliente": { "nombre": "Luis Paz" },
"total_pedido": 17662.15 }
]
// Total recuperados: 4 (IDs: 5, 6, 1, 2)
```

Justificación del Umbral de Importe: El umbral seleccionado fue de **$15.000,00**. En el conjunto muestral de Food Store, los pedidos presentan importes entre $3.175,70 y $25.104,04, con un ticket promedio de **$16.764,24**. El umbral de $15.000,00 discrimina con precisión los pedidos de consumo individual o colación rápida (como el Pedido 4 de snacks/congelados por $3.175,70) de aquellos correspondientes a **Compras Mayoristas o Eventos Gastronómicos** (Pedidos 1, 2, 5 y 6). En términos operacionales, superar este umbral califica al pedido para despacho prioritario con transporte refrigerado y asignación de gestor de cuenta.

---

2. Consultas Equivalentes en PostgreSQL (`psql` — Consigna 5)

Equivalente a Consigna 4 (b): Filtro por estado `'ENTREGADO'`

```
SELECT
    p.id_pedido, p.fecha_hora, p.forma_pago, p.estado, c.nombre AS cliente,
    round(SUM(dp.cantidad * dp.precio_unitario)::numeric, 2) AS total_pedido
FROM Pedido p
JOIN Cliente c ON p.id_cliente = c.id_cliente
JOIN Detalle_Pedido dp ON p.id_pedido = dp.id_pedido
WHERE p.estado = 'ENTREGADO'
  AND p.id_pedido BETWEEN 1 AND 6
GROUP BY p.id_pedido, p.fecha_hora, p.forma_pago, p.estado, c.nombre
ORDER BY p.id_pedido;
```

Salida de consola (psql):

```
 id_pedido |          fecha_hora           |    forma_pago     |  estado   |
cliente    | total_pedido
-----------+-------------------------------+-------------------+-----------+------
---------+--------------
         1 | 2026-09-21 22:22:22.22525-03  | EFECTIVO          | ENTREGADO | Ana
Gómez      |     17804.34
         2 | 2026-09-26 22:22:22.22525-03  | TARJETA           | ENTREGADO | Luis
Paz        |     17662.15
         4 | 2025-01-05 14:38:06.391101-03 | TARJETA           | ENTREGADO |
Cliente 7650  |      3175.70
         5 | 2026-08-29 08:38:48.485626-03 | BILLETERA_DIGITAL | ENTREGADO |
Cliente 19190 |     25104.04
(4 filas)
```

Equivalente a Consigna 4 (c): Pedidos con `id_producto = 3` ('Coca 1.5L')

```sql
SELECT DISTINCT
    p.id_pedido, p.fecha_hora, p.estado, c.nombre AS cliente,
    dp.id_producto, pr.nombre_producto, dp.cantidad, dp.precio_unitario,
    round((dp.cantidad * dp.precio_unitario)::numeric, 2) AS subtotal
FROM Pedido p
JOIN Cliente c ON p.id_cliente = c.id_cliente
JOIN Detalle_Pedido dp ON p.id_pedido = dp.id_pedido
JOIN Producto pr ON dp.id_producto = pr.id_producto
WHERE dp.id_producto = 3
  AND p.id_pedido BETWEEN 1 AND 6
ORDER BY p.id_pedido;
```

Salida de consola (`psql`):

```
 id_pedido |          fecha_hora           |     estado     |  cliente   |
id_producto | nombre_producto | cantidad | precio_unitario | subtotal
-----------+-------------------------------+----------------+------------+---------
----+-----------------+----------+-----------------+----------
         1 | 2026-09-21 22:22:22.22525-03 | ENTREGADO      | Ana Gómez  |
3 | Coca 1.5L       |        1 |          800.00 |   800.00
         3 | 2026-09-30 22:22:22.22525-03 | EN_PREPARACION | Marta Ruiz |
3 | Coca 1.5L       |        4 |          800.00 |  3200.00
(2 filas)
```

---

3. Tabla de Verificación Cruzada y Análisis Técnico

Consulta	Cantidad en PostgreSQL	Cantidad en MongoDB	Coinciden los IDs
Estado seleccionado (`ENTREGADO`)	**4**	**4**	**Sí** (IDs: 1, 2, 4, 5)
Producto seleccionado (`id_producto = 3`)	**2**	**2**	**Sí** (IDs: 1, 3)

Análisis de Correspondencia: No se detectaron discrepancias entre ambos motores.

- En la consulta por **Estado**, ambos motores identificaron unívocamente los 4 pedidos entregados con idénticos valores de cabecera y totales.
- En la consulta por **Producto**, en PostgreSQL fue indispensable ejecutar un `JOIN` entre `Pedido`, `Detalle_Pedido` y `Producto` aplicando la cláusula `DISTINCT` para proteger el resultado de duplicaciones si un pedido tuviese renglones múltiples. En contraste, en MongoDB la consulta se ejecutó de forma natural y sinérgica contra el arreglo embebido mediante la clave multikey `"lineas.id_producto"`, recuperando directamente la entidad completa en una sola pasada.

Parte 3 · Evaluación de la Tecnología e Inteligencia Artificial

1. Prompt de Consulta Enviado a la IA (Consigna 6)

```
"Actuá como Arquitecto de Datos Senior en bases de datos relacionales y NoSQL.
Necesito comparar PostgreSQL (nuestro motor relacional actual) y MongoDB (modelo
documental)
para gestionar el almacenamiento y la consulta del módulo de Pedidos y Detalles de
Pedido
de la plataforma Food Store.

Tené en cuenta las siguientes características de nuestro caso real:
1. Modelo actual: Contamos con tablas normalizadas (Pedido, Detalle_Pedido,
Cliente, Producto, Categoria)
   con restricciones de clave foránea, triggers de stock y borrado lógico con
columna 'eliminado'.
2. Patrón de acceso: La aplicación web y el centro de distribución consultan con
altísima frecuencia
   pedidos completos (cabecera + datos de contacto del cliente + totalidad de
líneas con subtotales
   y categoría para el armado de paquetes y facturación inmediata).
3. Requisitos de consistencia: El negocio no tolera desajustes en el stock
disponible, exige
   inmutabilidad estricta en los precios históricos pactados al momento de la
venta y coherencia
   matemática en los totales facturados ante concurrencia.

Evalúa pros, contras, rendimiento y recomendá fundamentadamente si conviene
mantener PostgreSQL,
migrar a MongoDB o adoptar una arquitectura híbrida."
```

2. Contraste Técnico de Afirmaciones de la IA (Consigna 7)

Afirmación 1 de la IA:

> *"MongoDB no ofrece garantías ACID reales en transacciones con múltiples entidades, por lo que es inviable garantizar la consistencia financiera de pedidos y líneas sin arriesgar pérdidas de datos o inconsistencias contables."*

- **Contraste con la Documentación Oficial de MongoDB y el Laboratorio: Afirmación Falsa y Desactualizada.** De acuerdo con el *MongoDB Manual (Core Operations - Transactions & Single-Document Atomicity)*:
  1. Desde MongoDB 4.0 (en réplicas) y 4.2 (en clusters fragmentados), el motor soporta formalmente **Transacciones Multi-Documento ACID** utilizando el protocolo de commit en dos fases (*Two-Phase Commit*) y aislamiento de tipo Snapshot.
  2. Más determinante aún para el modelo de Food Store: al estructurar el pedido bajo el patrón de **Document Embedding**, las líneas del pedido y la instantánea del cliente residen dentro del **mismo documento**. MongoDB garantiza **Atomicidad en Documento Único** (*Single-Document Atomicity*) sin necesidad de invocar transacciones complejas. La inserción, actualización o cancelación de un pedido y todas sus líneas ocurre de manera indivisible: o se guarda el pedido con sus 4 líneas y su `total_pedido` íntegro, o no se guarda nada, garantizando consistencia matemática inmediata a costo cero de bloqueo.

Afirmación 2 de la IA:

> *"Migrar todo a MongoDB siempre superará en velocidad a PostgreSQL porque elimina los JOINs y no requiere gestionar esquemas ni bloqueos."*

- **Contraste con la Documentación Técnica de PostgreSQL y Resultados de Laboratorio: Afirmación Falaz por Simplificación Excesiva.**
  1. Si bien el laboratorio demostró que la lectura de pedidos completos en MongoDB es altamente eficiente al evitar la unión en memoria de 4 tablas, delegar el 100% del sistema a MongoDB introduce severos inconvenientes operativos. El decremento atómico de stock (`Producto.stock`) ante una venta simultánea requiere modificar documentos en dos colecciones disjuntas (`pedidos` y `productos`). En PostgreSQL, este proceso se resuelve nativamente en milisegundos mediante el motor de concurrencia MVCC, triggers procedimentales PL/pgSQL (`fn_validar_stock_pedido`) y bloqueos a nivel de fila `SELECT ... FOR UPDATE`.
  2. Asimismo, en reportes analíticos agregados (como la vista materializada `mv_facturacion_categoria_mes` probada en TP5), PostgreSQL ejecuta escaneos indexados B-Tree y paralelismo nativo con menor huella de memoria RAM que los pipelines de `$unwind` y `$group` del Aggregation Framework de MongoDB sobre grandes volúmenes.

---

3. Conclusión Arquitectónica Fundamentada (Límite: 200 palabras)

Para el subsistema de pedidos de Food Store, la arquitectura óptima es un **esquema híbrido** (*Polyglot Persistence*). **PostgreSQL** debe consolidarse como la Fuente Única de la Verdad (*Single Source of Truth*) transaccional OLTP. Su motor MVCC, integridad referencial por claves foráneas, restricciones de dominio y

triggers de concurrencia (`fn_validar_stock_pedido`) aseguran la reserva atómica de stock y la inmutabilidad contractual de los precios de venta.

En paralelo, **MongoDB** actúa como almacén de lectura optimizado y proyección de agregados (*Read Model* en patrón CQRS). Al almacenar los pedidos completos como documentos autosuficientes con sus líneas y datos del cliente embebidos, se maximiza la **localidad espacial de referencia**, permitiendo al frontend web y a logística recuperar comprobantes en milisegundos mediante una sola operación de lectura sin recurrir a costosos `JOIN` ni penalizar las tablas operativas de PostgreSQL.